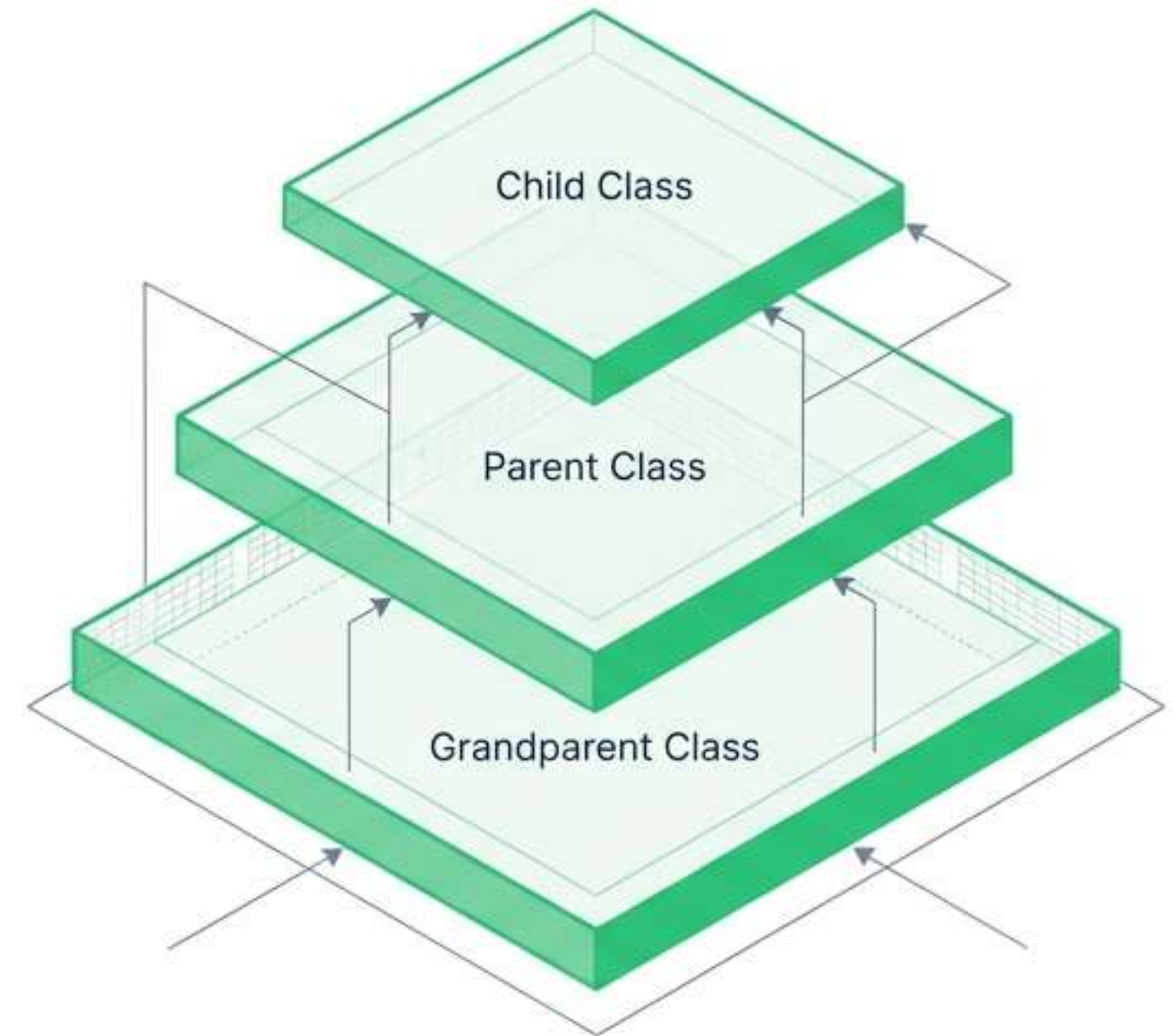


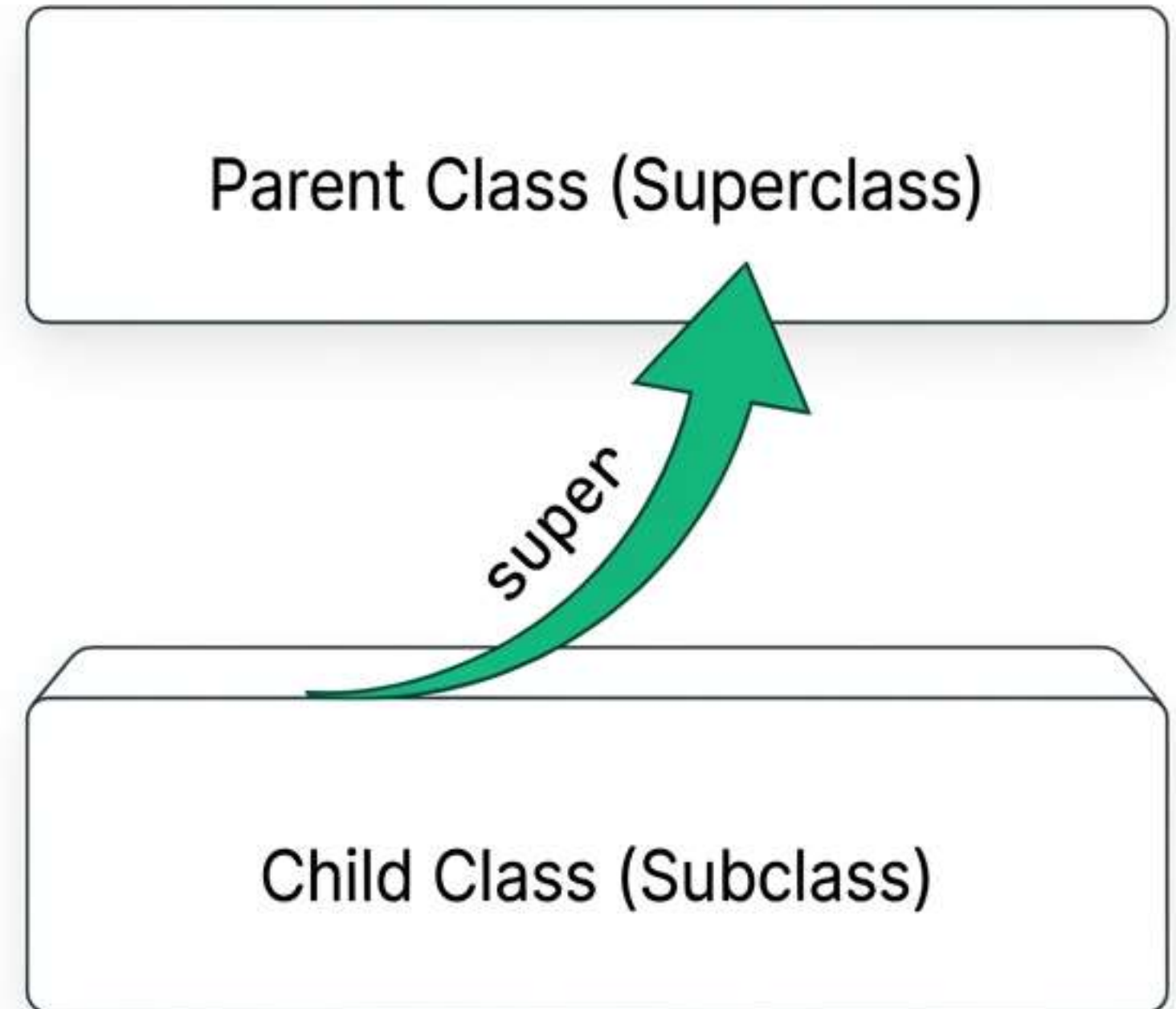
Mastering Java Inheritance: The **super** Keyword

Advanced Object-Oriented Programming for Test Automation Frameworks

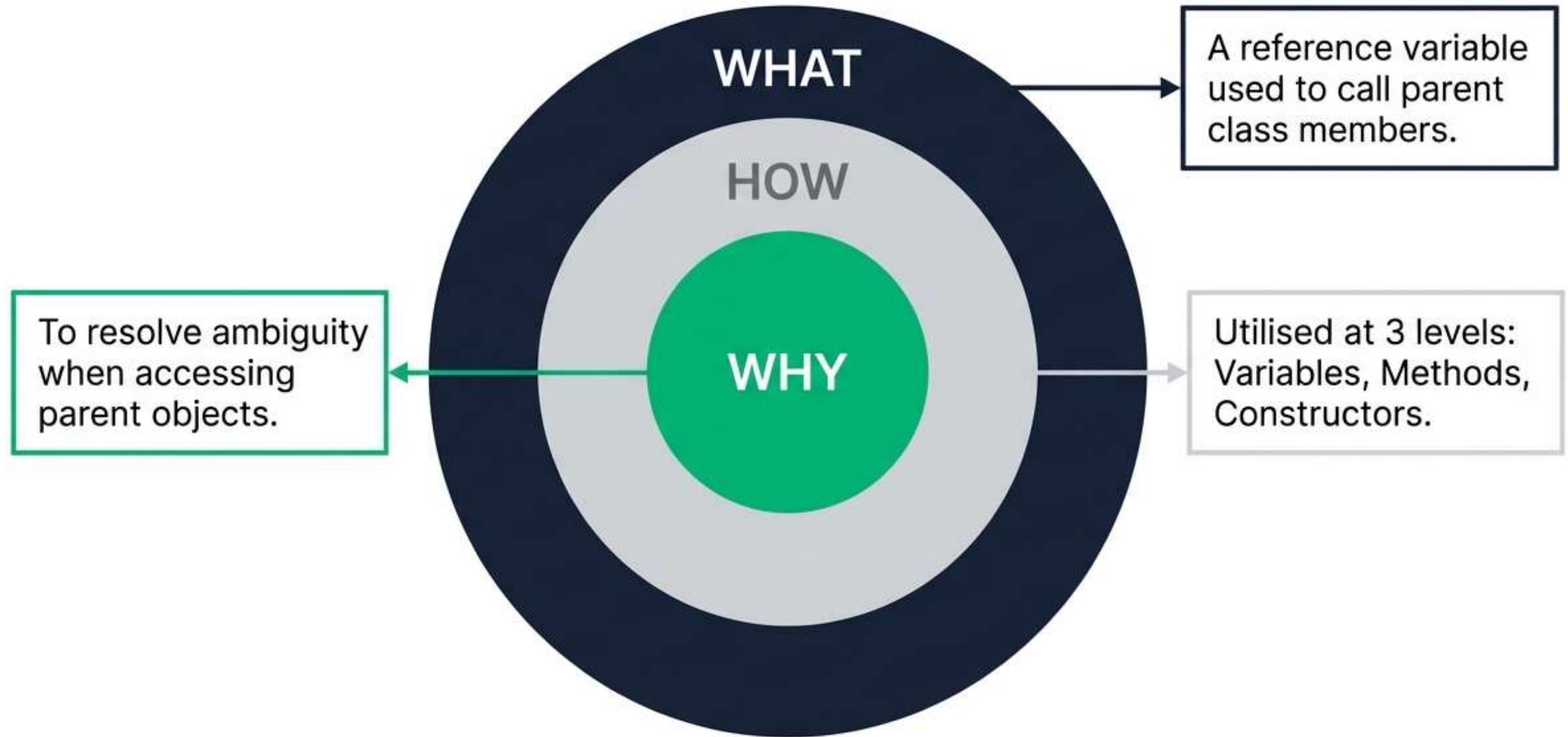


The Role of Context in Inheritance

- **The Context:** Classes inherit properties from parents to maximise code reusability.
- **The Problem:** When Subclass and Superclass share member names, ambiguity arises.
- **The Solution:** 'super' acts as a reference bridge, explicitly bypassing local scope.
- **Key Value:** Essential for robust test frameworks.



Strategic Framework: What, Why, How



Mechanism 1: Handling Variable Shadowing

Parent Class (Vehicle)

```
public class Vehicle {  
    String brakeModel = 'Disc';  
}
```

Child Class (Car)

```
public class Car extends Vehicle {  
    String brakeModel = 'ABS';  
  
    void printDetails() {  
        System.out.println(super.brakeModel);  
        // Outputs: Disc  
    }  
}
```

explicitly accesses parent's shadowed variable

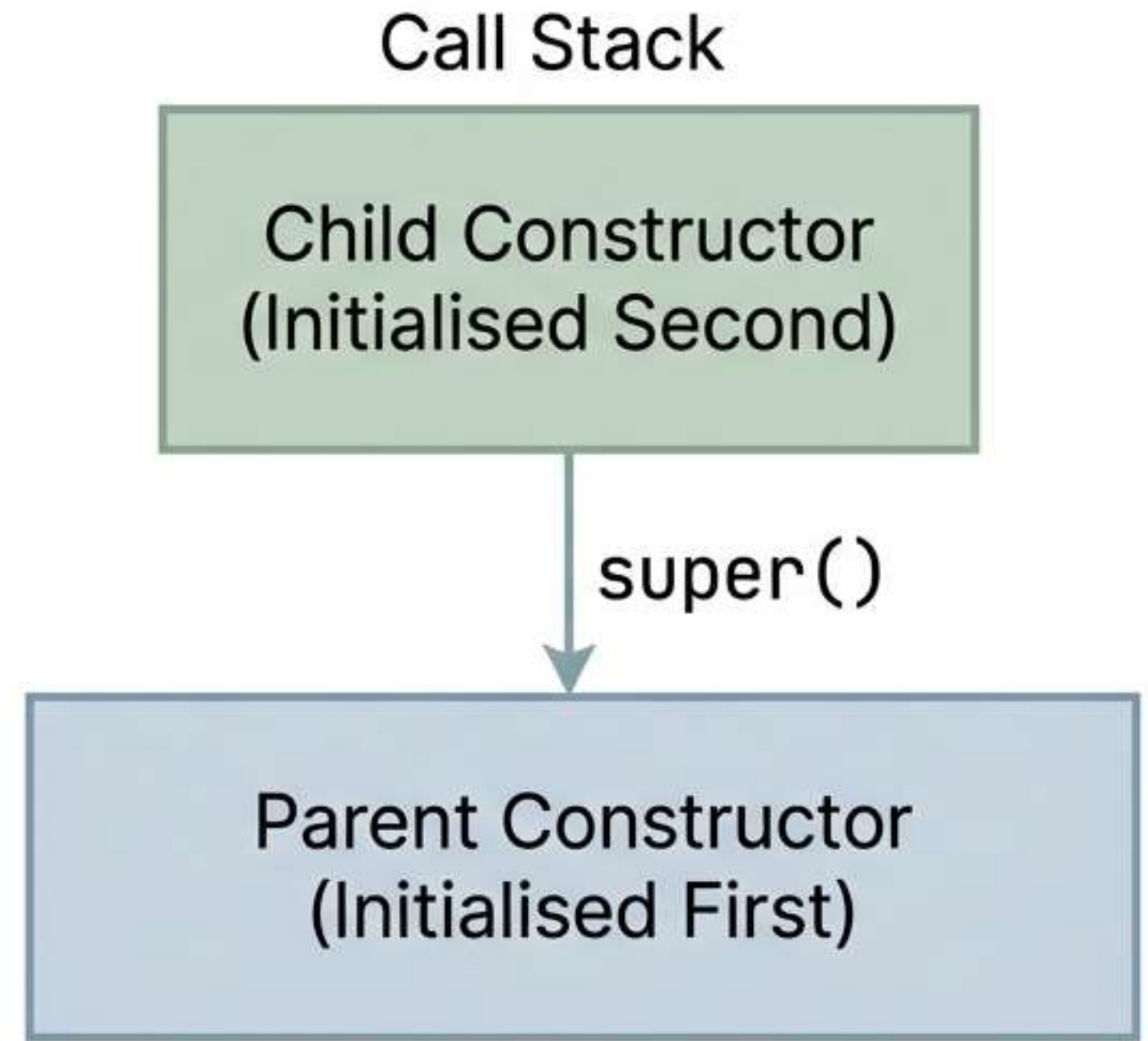
Mechanism 2: Invoking Overridden Methods



Benefit: Enables code extension rather than code replacement.

Mechanism 3: Constructor Chaining

- Constructors are not inherited, but invoked.
- Mechanism: `super()` calls the parent constructor.
- The Golden Rule: `super()` must be the FIRST statement in the child constructor.



Practical Implementation: Vehicle Systems

```
1 class Car extends Vehicle {  
2     String brakeModel = 'ABS';  
3  
4     public void displayFeatures() {  
5         // Variable Access  
6         System.out.println(super.brakeModel); // Output: Disc  
7         System.out.println(this.brakeModel);  // Output: ABS  
8  
9         // Method Access  
10        super.brandName('Hyundai'); // Runs Parent Logic  
11    }  
12 }
```

Console Output

```
> Disc  
> ABS  
> From Vehicle Class Hyundai
```

Architecture Pitfalls & Constraints



The First Statement Rule

`super()` must always be **line #1** in a constructor. Placing it second causes a **compilation error**.



The Static Limitation

`super` refers to an instance. It **CANNOT** be used inside **static contexts** (like `main`) because static methods belong to the class, not the object.



Single Inheritance

Java **does not support multiple inheritance**. `super` always refers to the **one immediate parent**.

Executive Summary



Scope: 'super' is the specific bridge to the immediate parent class.



Versatility: Resolves name collisions in Variables and Methods.



Initialisation: Critical for chaining logic in Constructors.



Constraint: Cannot be used in static contexts (like main).



Application: Essential for robust inheritance in test automation.